SOLVED ASSIGNMENT

AWS EC2 Terraform Controller & Nested Infrastructure Deployment

Aim: Create an EC2-based Terraform controller/bastion using IAM Instance Profile authentication, automatically install Terraform with user_data, run a second Terraform workspace inside the EC2 instance, provision a nested S3 bucket, and perform complete teardown.

This solution follows the uploaded 6-page lab guide. The source describes the architecture as a Controller/Bastion Pipeline with outer Terraform running locally and inner Terraform running from the EC2 controller.

1. ARCHITECTURE & EXECUTION FLOW

```
Local Workstation
      ■
      ■ terraform apply
      ▼
AWS EC2 Terraform Controller
      ■
      ■■■ IAM Instance Profile
      ■■■ user_data installs Terraform
      ■■■ ~/secondary-infra/main.tf
              ■
              ■ terraform init / plan / apply
              ▼
      Secondary Infrastructure
              ■
              ■■■ S3 Bucket
              ■
              ▼
       AWS STS Temporary Credentials
```

Core principles from the guide:

Zero credentials on disk: EC2 uses temporary credentials from the AWS Instance Metadata Service (IMDSv2).

Fully automated bootstrap: user_data updates Ubuntu, adds the HashiCorp repository, installs Terraform and creates the nested Terraform project.

Dual-stage lifecycle: Outer Terraform creates the runner node; Inner Terraform provisions targeted cloud resources from that node.

2. PART 1 – CREATE THE OUTER TERRAFORM PROJECT

Create the local project directory and Terraform files:

```
mkdir -p ~/terraform-ec2-controller && cd ~/terraform-ec2-controller
touch main.tf variables.tf outputs.tf
```

variables.tf

```
variable "aws_region" {
  type        = string
  description = "AWS deployment region"
  default     = "us-east-1"
}

variable "instance_type" {
  type        = string
  description = "EC2 instance sizing"
  default     = "t3.micro"
}
```

3. main.tf – TERRAFORM VERSION, PROVIDERS & AWS PROVIDER

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.40"
    }

    tls = {
      source  = "hashicorp/tls"
      version = "~> 4.0"
    }
```

```
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project   = "Terraform-Controller-Lab"
      ManagedBy = "Terraform-Outer"
    }
  }
}
```

4. IAM ROLE & INSTANCE PROFILE

The EC2 instance receives an IAM role through an instance profile. This lets Terraform inside EC2 call AWS services without hardcoded access keys.

```
resource "aws_iam_role" "terraform_runner_role" {
  name = "terraform-controller-ec2-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "admin_attach" {
  role       = aws_iam_role.terraform_runner_role.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}

resource "aws_iam_instance_profile" "runner_profile" {
  name = "terraform-controller-instance-profile"
  role = aws_iam_role.terraform_runner_role.name
}
```

The lab guide uses AdministratorAccess for demonstration. For production, it explicitly recommends replacing this with a narrowly scoped least-privilege policy.

5. AUTOMATED RSA SSH KEY PAIR

```
resource "tls_private_key" "ssh_key" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

resource "aws_key_pair" "generated_key" {
  key_name   = "tf-runner-key"
  public_key = tls_private_key.ssh_key.public_key_openssh
}

resource "local_file" "private_key_pem" {
  content         = tls_private_key.ssh_key.private_key_pem
  filename        = "${path.module}/tf-runner-key.pem"
  file_permission = "0400"
}
```

6. SECURITY GROUP CONFIGURATION

```
data "aws_vpc" "default" {
  default = true
}

resource "aws_security_group" "runner_sg" {
  name        = "terraform-runner-sg"
  description = "Allow inbound SSH access"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "SSH administrative ingress"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow outbound traffic for downloading Terraform packages & APIs"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

The lab opens SSH to 0.0.0.0/0. The guide's production hardening section recommends restricting SSH to a corporate/VPN IP or using AWS Systems Manager Session Manager.

7. UBUNTU AMI LOOKUP

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

8. EC2 CONTROLLER INSTANCE

```
resource "aws_instance" "terraform_runner" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = var.instance_type
  key_name               = aws_key_pair.generated_key.key_name
  vpc_security_group_ids = [aws_security_group.runner_sg.id]
  iam_instance_profile   = aws_iam_instance_profile.runner_profile.name
  associate_public_ip_address = true

  root_block_device {
    volume_size           = 20
    volume_type           = "gp3"
    delete_on_termination = true
  }

  # user_data bootstrap is added here
}
```

9. USER_DATA – AUTOMATIC TERRAFORM BOOTSTRAP

The EC2 startup script performs the following tasks: updates Ubuntu, installs prerequisites, adds the official HashiCorp repository, installs Terraform, creates the secondary Terraform workspace, and assigns it to the ubuntu user.

```
user_data = <<-EOF
#!/bin/bash
set -e

export DEBIAN_FRONTEND=noninteractive
apt-get update -y
apt-get install -y gnupg software-properties-common curl git unzip

wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | te
```

```
apt-get update -y
apt-get install -y terraform

mkdir -p /home/ubuntu/secondary-infra

# Secondary Terraform main.tf is created here.

chown -R ubuntu:ubuntu /home/ubuntu/secondary-infra
EOF
```

10. INNER / SECONDARY TERRAFORM PROJECT

The EC2 controller creates ~/secondary-infra/main.tf. This is the Terraform project that runs inside EC2 and provisions the nested AWS resource.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }

    random = {
      source  = "hashicorp/random"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "${var.aws_region}"
}

resource "random_id" "bucket_suffix" {
  byte_length = 4
}

resource "aws_s3_bucket" "test_bucket" {
  bucket       = "nested-tf-bucket-$${random_id.bucket_suffix.hex}"
  force_destroy = true

  tags = {
    CreatedBy   = "EC2-Terraform-Runner"
    Environment = "Nested-Lab"
  }
}

output "created_bucket_name" {
  value = aws_s3_bucket.test_bucket.bucket
}
```

11. outputs.tf

```
output "controller_public_ip" {
  description = "Public IP address of the Terraform controller EC2 instance"
  value       = aws_instance.terraform_runner.public_ip
}

output "ssh_command" {
  description = "Ready-to-use SSH connection command"
  value       = "ssh -i ${local_file.private_key_pem.filename} ubuntu@${aws_instance.terraform_runner.public_ip}"
}
```

12. PART 2 – DEPLOY THE CONTROLLER FROM LOCAL MACHINE

Run these commands from the local Terraform project:

```
terraform init
terraform plan
terraform apply -auto-approve
```

Expected lab result shown in the source: **Apply complete! Resources: 6 added, 0 changed, 0 destroyed.**

> The guide notes that approximately 60–90 seconds may be required after terraform apply for cloud-init/user_data to finish installing Terraform on EC2.

13. CONTROLLER OUTPUTS

```
controller_public_ip = "<EC2-PUBLIC-IP>"
ssh_command = "ssh -i ./tf-runner-key.pem ubuntu@<EC2-PUBLIC-IP>"
```

Use the actual IP printed by your own Terraform run rather than relying on the example IP shown in the guide.

14. PART 3 – RUN TERRAFORM INSIDE EC2

Step 4.1: Connect to the controller

```
ssh -i ./tf-runner-key.pem ubuntu@$(terraform output -raw controller_public_ip)
```

Step 4.2: Verify Terraform

```
terraform -version
```

The guide shows an expected example of Terraform v1.8.x on linux_amd64.

Step 4.3: Open the secondary workspace

```
cd ~/secondary-infra
ls -la
cat main.tf
```

Step 4.4: Initialize and Apply Nested Infrastructure

```
terraform init
terraform plan
terraform apply -auto-approve
```

The AWS provider automatically uses the EC2 Instance Profile. No AWS access key or secret key is entered into the Terraform project.

The guide's successful run shows **Resources: 2 added, 0 changed, 0 destroyed** and an output named **created_bucket_name**.

15. IAM AUTHENTICATION FLOW

```
Terraform running inside EC2
        ↓
AWS Provider
        ↓
EC2 Instance Profile
        ↓
AWS STS / Instance Metadata Service
        ↓
Temporary Security Credentials
        ↓
S3 Bucket Provisioned
```

This is the key concept of the assignment: the EC2 controller authenticates through its attached IAM role rather than storing permanent AWS keys.

16. STEP 4.5 – DECOMMISSION NESTED INFRASTRUCTURE

```
terraform destroy -auto-approve
exit
```

The nested S3 bucket should be destroyed before terminating the controller.

17. PART 4 – COMPLETE TEARDOWN FROM LOCAL MACHINE

```
terraform destroy -auto-approve
```

The source guide's teardown order is:

Order	Action
1	Terminate the EC2 controller instance.
2	Delete the IAM instance profile and detach the role policy.
3	Delete the security group and AWS EC2 key pair.
4	Delete the local tf-runner-key.pem file.

Expected final output in the guide: **Destroy complete! Resources: 6 destroyed.**

18. PRODUCTION HARDENING & BEST PRACTICES

Domain	Lab Practice	Production Recommendation
IAM Permissions	AdministratorAccess attached to role	Follow least privilege; scope permissions to required resources such as S3/
State Management	Local state on EC2 runner disk	Use remote S3 backend with DynamoDB state locking to persist state acros
Network Ingress	SSH 22 open to 0.0.0.0/0	Restrict to corporate VPN IP or use AWS Systems Manager Session Manag
Instance Metadata	Default IMDSv1/v2 enabled	Enforce IMDSv2 using http_tokens = "required" to mitigate SSRF attacks.

19. COMPLETE COMMAND CHEAT SHEET

```
# Local machine
mkdir -p ~/terraform-ec2-controller && cd ~/terraform-ec2-controller
touch main.tf variables.tf outputs.tf

terraform init
terraform plan
terraform apply -auto-approve

# Connect to EC2
ssh -i ./tf-runner-key.pem ubuntu@$(terraform output -raw controller_public_ip)

# Inside EC2
terraform -version
cd ~/secondary-infra
ls -la
cat main.tf

terraform init
terraform plan
terraform apply -auto-approve

# Clean nested infrastructure
terraform destroy -auto-approve
exit

# Local machine: destroy controller
terraform destroy -auto-approve
```

20. VIVA QUESTIONS & ANSWERS

Q1. What is the main idea of this lab?
An EC2 instance acts as a Terraform controller/bastion and runs a second Terraform workspace inside itself.

Q2. Why use an IAM Instance Profile?
It allows EC2 to obtain temporary AWS credentials without storing permanent access keys on disk.

Q3. What is outer Terraform?
Terraform running on the local workstation that creates the EC2 controller and its supporting AWS resources.

Q4. What is inner Terraform?
Terraform running inside the EC2 controller to provision the secondary infrastructure.

Q5. What does user_data do?
It bootstraps the EC2 instance by installing prerequisites, Terraform, and creating the secondary Terraform project.

Q6. What resource is created by the nested Terraform project?
An S3 bucket with a random suffix.

Q7. What is STS?
AWS Security Token Service provides temporary security credentials used by the EC2 role.

Q8. Why is AdministratorAccess not recommended for production?
It grants broad permissions. Production systems should follow least-privilege IAM.

Q9. What is terraform init?
It initializes the Terraform working directory and downloads required provider plugins.

Q10. What is terraform plan?
It previews the infrastructure changes Terraform intends to make.

Q11. What is terraform apply?
It applies the planned configuration and creates/changes infrastructure.

Q12. What is terraform destroy?
It removes infrastructure managed by the Terraform configuration.

21. RESULT

The AWS EC2 Terraform Controller lab was completed conceptually as specified: outer Terraform provisions an EC2 controller with an IAM Instance Profile, cloud-init installs Terraform and creates a secondary workspace, and inner Terraform uses the EC2 role to provision a nested S3 bucket without hardcoded AWS credentials. The nested infrastructure and controller are then destroyed using Terraform.

Important: The example public IP and bucket name printed in the source are demonstration outputs. For an actual lab submission, replace them with the values produced by your own Terraform execution.

— END OF SOLVED ASSIGNMENT —